

DESARROLLO DE APLICACIONES MÓVILES CON IA

NGD Tech Solutions

Enfoque del Servicio

Este chat será tu **base de operaciones** para:

-  **Diseño y desarrollo de Apps Móviles** (Android primero, iOS cuando toque)
 -  **Aplicaciones con IA integrada**
(apps que “entienden” documentos, ideas, prompts, flujos)
 -  **Adaptación de Apps Móviles a Web Apps**
(WebView, PWA, Firebase, backends compartidos)
 -  **Aprender desarrollo asistido por IA**
(Cursor, prompts técnicos, generación rápida de apps)
 -  **Soporte, QA y mantenimiento profesional**
 -  **Publicación, documentación y visibilidad profesional**
-

Todo con:

-  **Nivel junior realista**
-  **Pero con arquitectura, mentalidad y calidad profesional**
-  **Pensando en empleabilidad, freelance y producto real**

Metodología de proyectos

Trabajaremos **por proyectos enumerados**

Estructura fija de cada proyecto

Cuando tú digas “*Proyecto #X*”, yo entregaré:

1. **Título del proyecto**
2. **Descripción clara del producto**
3. **Análisis técnico + opinión honesta**
4. **Encaje realista según tu perfil**
5. **Riesgos reales (técnicos, tiempo, mercado)**
6. **Análisis de mercado y necesidad**
7. **Stack recomendado (mobile, web, IA)**
8. **Plan de desarrollo por fases**
9. **Estrategia de documentación**
 - GitHub
 - LinkedIn
 - Portfolio
10. **Guía básica de marketing y visibilidad**
11. **Siguientes pasos**
12.  **Esperar siempre tu visto bueno antes de avanzar**

Tu perfil

Tu perfil es muy bueno para este enfoque, porque tienes:

- Base real en **Java / Kotlin / Android**
- Experiencia **real** (freelance, NTT DATA, proyectos vivos)
- QA → eso te da ventaja brutal en apps estables
- Unreal → mentalidad de sistemas y lógica
- Ya estás usando **IA como herramienta**, no como muleta

 Riesgo actual:

- Querer abarcar demasiado sin cerrar productos
 - Falta de estructura “producto → publicación → visibilidad”
-

Plan diario

- Resolver **dudas técnicas paso a paso**
- Explicar **por qué se hace algo**, no solo el cómo
- Dar alternativas:
 - *junior-friendly*
 - *profesional*
- Ayudarte a:
 - pensar cómo dev
 - pensar como QA
 - pensar como producto
- **Usar la IA para acompañar tus proyectos**

✓ Estructura de Proyectos (No se toca, caso omiso a esta página)

Opción A

👉 Proyecto #1 – App móvil con IA desde cero
(idea + arquitectura + Firebase + IA + publicación)

Opción B

👉 Proyecto #1 – Convertir una de tus apps actuales en App + Web
(APP CVC o NGD Studios)

Opción C

👉 Proyecto #1 – App “le das un documento y te crea la app”
(más ambicioso, muy alineado con IA)

Opción D

👉 Proyecto #1 – Fortalecer perfil profesional con un proyecto top
(pensado para GitHub + LinkedIn + empleo)

📌 Fecha Lanzamiento

La fecha estimada para publicar la marca: **MAYO - JUNIO**

Después de publicar la marca se anuncia el producto **NGD Tech Solutions**

- Con vídeo de presentación
- Vídeo del desarrollo
- Carteles
 - Logo, Logotipo y Eslogan
- Redes Sociales (Portfolio)
- Google Play

Primera APP - Soporte Técnico

APP de Soporte Técnico con IA, donde:

- Das soporte a clientes
- Puedes:
 - hablar con ellos (chat humano)
 - ofrecer un chat con IA
 - publicar novedades / cambios de sus apps
- Cada cliente ve:
 - el estado de *su* app
 - los cambios realizados
 - avisos y noticias

 **Zendesk / Freshdesk / Notion + IA**, pero versión indie y personalizada.



MODELO DE DATOS (Firestore)



MODELO DE DATOS (Firestore)

Esto es el esqueleto de todo.

pgsql

Copiar código

users/

{userId}

role: "admin" | "client"

name

email

apps: [appId]

apps/

{appId}

name

status

clientId

lastUpdate

updates/

{updateId}

appId

title

description

date

type: "update" | "notice"

chats/

{chatId}

appId

clientId

type: "human" | "ai"

lastMessage

updatedAt

messages/

{messageId}

chatId

sender: "client" | "admin" | "ai"

text

timestamp



Competencias

1. Arquitectura de Slack

Slack usa una arquitectura **cliente-servidor basada en microservicios** diseñada para mensajería en tiempo real y alta escalabilidad.

Componentes principales

1. Clientes

- Apps de **Web, Desktop y Mobile**.
- Todos se conectan al backend mediante **HTTPS y WebSockets**.

2. Gateway / Edge Layer

- Primer punto de entrada.
- Maneja:
 - autenticación
 - rate limiting
 - routing de peticiones.

3. Sistema de Mensajería en Tiempo Real

- Usa **WebSockets** para mantener conexiones persistentes.
- Permite:
 - entrega inmediata de mensajes
 - presencia online
 - typing indicators.

4. Microservicios

Slack separa funciones en servicios independientes:

Ejemplos:

- servicio de mensajes
- servicio de canales
- servicio de usuarios
- servicio de archivos
- servicio de notificaciones

Esto permite:

- desplegar cambios sin afectar todo el sistema
- escalar solo los servicios con mayor carga.

5. Sistema de colas

Usan colas internas (similar a Kafka) para:

- procesar eventos
- distribuir mensajes
- sincronizar notificaciones.

6. Almacenamiento

Diferentes bases de datos según el tipo de datos:

- **MySQL** → metadata
- **Redis / Memcache** → cache
- **S3-like storage** → archivos

7. Search Service

Slack tiene un servicio separado para **indexación y búsqueda de mensajes**.

Flujo simplificado de un mensaje

1. Usuario escribe mensaje.
2. Cliente lo envía por **WebSocket**.
3. Gateway lo recibe.
4. Servicio de mensajes lo procesa.
5. Se guarda en base de datos.
6. Se publica evento en cola.
7. Se envía en tiempo real a los clientes conectados del canal.

2. Arquitectura de Discord

Discord está optimizado para **chat masivo + voz + gaming**.

Su arquitectura es más orientada a **eventos distribuidos**.

Componentes principales

1. Cliente

- Web
- Desktop (Electron)
- Mobile

Usan **HTTP + WebSocket**.

2. Gateway Servers

Uno de los componentes clave. Funciones:

- mantener **millones de WebSockets**
- manejar eventos:
 - mensajes
 - reacciones
 - presencia
 - voice states

Los usuarios se conectan a diferentes **gateways distribuidos**.

3. API Servers

Procesan operaciones como:

- enviar mensajes
- crear canales
- modificar servidores
- gestionar permisos

Usan arquitectura de **microservicios en Python / Go**.

4. Event Bus

Discord usa un **bus de eventos (Kafka)** para sincronizar acciones entre servicios.

Ejemplo:

mensaje enviado

- → evento en Kafka
- → servicios lo consumen
- → notificaciones
- → almacenamiento
- → analytics

5. Almacenamiento

- **Cassandra** → mensajes (altísima escala)
- **Redis** → cache
- **ElasticSearch** → búsqueda
- **S3** → archivos

6. Sistema de voz

La voz es **otro sistema separado**. Componentes:

- Voice Gateway
- Voice UDP servers
- WebRTC

Esto permite:

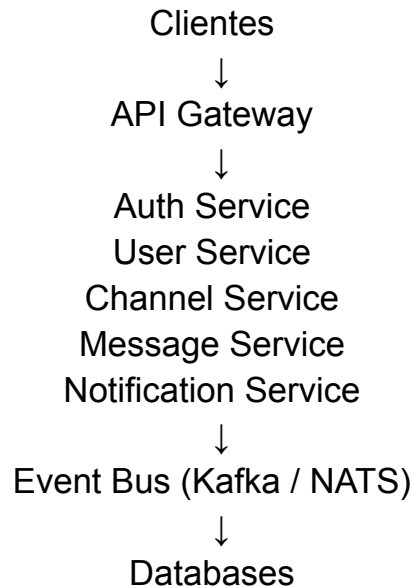
- baja latencia
- millones de usuarios en voz simultánea.

3. Diferencias clave Slack vs Discord

Aspecto	Slack	Discord
Objetivo	Empresas	Comunidades / gaming
Mensajes	MySQL	Cassandra
Tiempo real	WebSockets	WebSockets
Eventos	Colas internas	Kafka
Voz	Limitada	Sistema delicado
Escala de canales	Menor	Millones

4. Arquitectura típica si queremos construir nuestra propia app tipo Slack/Discord

La arquitectura recomendada sería:



Y añadir:

- WebSocket Gateway
- Message Queue
- Search Service
- Media Storage

FASES DEL PROYECTO

FASE 1 — BASE TÉCNICA (✓ ya hecha)

- Gradle estable
 - Firebase conectado
 - Proyecto sano
-

FASE 2 — AUTENTICACIÓN Y ROLES

- Login / Register
- Persistencia de sesión
- Roles (admin / cliente)
- Redirección inteligente

 Sin esto nada tiene sentido

FASE 3 — FIRESTORE & MODELOS

- Crear modelos Kotlin
 - Repositorios
 - Reglas de seguridad
 - Datos en tiempo real
-



FASE 4 — ESTADO DE LA APP DEL CLIENTE

- Cada cliente ve:
 - estado
 - progreso
 - última actualización
 - Vista clara tipo dashboard
-



FASE 5 — NOVEDADES / AVISOS

- Admin pública noticias
 - Cliente las ve en tiempo real
 - Histórico
 - Notificaciones (push)
-



FASE 6 — CHAT HUMANO

- Chat en tiempo real
 - Soporte humano
 - Historial
 - Varios chats por app
-



FASE 7 — CHAT CON IA

- Endpoint seguro
 - Contexto por app
 - Historial resumido
 - Respuestas útiles
 - Fallback a humano
-
- Modularización futura
 - Multi-empresa escalable
 - Optimización consultas
 - Cache offline
 - Performance real
 - UI profesional
 - Testing real
 - Preparación producción



FASE 8 — SEGURIDAD

- Reglas Firestore
- Validaciones
- Protección IA
- Roles estrictos

FASE 9 — PREPARACIÓN WEB

- Arquitectura compartida
 - Firestore común
 - Futuro panel web admin
 - Multi-plataforma
-

FASE 10 — PULIDO FINAL

- UX
- Estados vacíos
- Errores controlados
- Escalabilidad

Dentro del chat humano y con IA

- Dime tu nombre
- Nombre de tu empresa, negocio o aplicación
- Qué necesitas (Cuál es tu problema, en qué te puedo ayudar)

Ejecución - Fases del Proyecto

1 FASE 1 - Base técnica | Dashboard funcional

Se cargan negocios según rol

ADMIN ve todos los negocios de su empresa

CLIENT y **SOPORTE** ven solo su negocio

Se muestran:

- name
- status
- lastUpdate

Arquitectura **MVVM** real:

- DashboardViewModel
- Repository
- LiveData
- RecyclerView con **AppAdapter**

2 FASE 2 - Autenticación

- Login con Firebase Authentication
- Usuarios reales en colección **users**
- Roles funcionando:
 - **ADMIN** - Cada empresa o cliente tiene un usuario admin para ver toda la aplicación
 - **CLIENT** - Es la empresa o cliente que puede acceder para ver el estado de su aplicación
 - **SOPORTE** (admin@ngd.com)

3 FASE 3 - Estructura en Firestore / Firebase

- users
- companies
- apps (legacy / fallback)
- updates (aún sin usar completamente)

Flujo de Firebase

- User → companyId → businesses (companies/{companyId}/businesses)

Seguridad lógica

Métodos

- `canCreateBusinesses()`
- `canPublishUpdates()`
- `canRespondChats()`

4FASE 4 - Estado de la App de cliente

📌 Estado completo del proyecto

- En desarrollo
- En revisión
- En producción
- En mantenimiento
- Pausado

- 🔧 BLOQUE 1 — Mejorar estructura de Firestore
- 🔧 BLOQUE 2 — Actualizar BusinessModel
- 🔧 BLOQUE 3 — Actualizar conversión a AppModel
- 🔧 BLOQUE 4 — Mejorar layout del item
- 🔧 BLOQUE 5 — Indicadores visuales dinámicos
- 🔧 BLOQUE 6 — Actualización automática de `lastUpdate`

 CLIENTE	 ADMIN	 NEGOCIO
• Estado real de su proyecto • Progreso visual • Versión • Tipo de soporte • Fecha real de actualización	• Todos sus negocios • Estado de cada uno • Nivel de avance global • Control profesional tipo CRM ligero	• Plataforma profesional de seguimiento de soporte y desarrollo para múltiples empresas.
EN FUNCIONAMIENTO		
➤ Estructura Firestore profesional ➤ BusinessModel completo ➤ AppModel ampliado ➤ Conversión correcta en ViewModel	➤ Dashboard visual estilo SaaS ➤ Colores dinámicos ➤ Iconos por estado	➤ Badge profesional ➤ Progreso protegido (0–100) ➤ Filtrado de inactivos ➤ Método preparado para actualizar lastUpdate

FASE 5 - Novedades y avisos

El cliente verá:

- Lista cronológica de novedades
- Tipo visual (colores por tipo)
- Fecha automática
- Histórico completo

El admin podrá:

- Crear novedades
- Editarlas
- Eliminar si es necesario

♦ **BLOQUE 1** - Estructura Firestore updates - Crear la subcolección correcta dentro de cada business.

♦ **BLOQUE 2** - Crear UpdateModel en Android - Modelo de datos alineado con Firestore.

♦ **BLOQUE 3** - Crear UpdatesRepository

Lógica para:

- Obtener updates
- Crear Updates
- Eliminar Updates
- Ordenador por fecha

 ¿Para qué sirve el **type**?

Sirve para:

- Cambiar color del update
- Mostrar icono diferente
- Filtrar por tipo
- Diferenciar gravedad/importancia

📌 Valores que vamos a usar

Nosotros definimos 4 tipos estándar:

type	Significado	Ejemplo
info	Información general	"Inicio del desarrollo"
mejora	Nueva funcionalidad	"Añadido sistema de reservas"
incidencia	Problema detectado	"Error en login solucionado"
versión	Nueva versión	"Versión 1.1 publicada"

◆ 5 ¿Dónde sí usarás .uid?

Cuando publiques un update.

Ejemplo futuro:

```
kotlin
```

```
val uid = FirebaseAuth.getInstance().currentUser?.uid ?: ""
```

Copiar código

Y eso se guarda en:

```
nginx
```

```
createdBy
```

Copiar código

“Si hay 3 devs, necesito 3 createdBy en el mismo update” **No.** Necesitas 3 updates distintos si publican 3 cosas distintas.

ESCENARIO REAL

Esa empresa tiene:

- 3 desarrolladores
- 1 jefe técnico
- 1 cliente final

Todos usan tu app. Todos pertenecen a la misma empresa.

```
users
├── UID_DEV_1
│   │ role: "Admin"
│   │ companyId: "EmpresaX"
│
├── UID_DEV_2
│   │ role: "Admin"
│   │ companyId: "EmpresaX"
│
├── UID_DEV_3
│   │ role: "Admin"
│   │ companyId: "EmpresaX"
│
└── UID_CLIENTE
    │ role: "Client"
    │ companyId: "EmpresaX"
```

¿Cómo publican updates?

companies/EmpresaX/businesses/Restaurante_madrid/updates

Cada vez que alguien publica: Se crea un documento nuevo.

```
updates
├── update_1
│   │ title: "Inicio backend"
│   │ createdBy: UID_DEV_1
│
├── update_2
│   │ title: "API completada"
│   │ createdBy: UID_DEV_2
│
└── update_3
    │ title: "Login corregido"
    │ createdBy: UID_DEV_3
```



Un update = una acción concreta.

Esa acción la hace una persona.

Si trabajan 3 personas → habrá 3 updates distintos.

Eso es correcto y profesional.

♦ **BLOQUE 4** - UpdatesViewModel + UiState - Arquitectura MVVM profesional para novedades.

- Cargue los updates desde Firebase
- Controle loading
- Controle errores
- Mantenga arquitectura limpia (MVVM)
- Prepare la pantalla visual

♦ **BLOQUE 5** - UpdatesActivity + RecyclerView

Pantalla visual:

- Lista tipo timeline
- Colores por tipo
- Iconos por tipo

Crear pantalla Updates (RecyclerView + Adapter)

- Crear UpdatesActivity
- Crear layout
- Crear UpdateAdapter
- Conectar con UpdatesViewModel
- Mostrar los updates reales en pantalla

♦ **BLOQUE 6** - Sistema para publicar Updates (Solo **ADMIN**) - Publicación desde ADMIN + actualización automática de lastUpdate

- Formulario para crear novedad
- Solo visible para ADMIN
- Al publicar → actualiza business.lastUpdate automáticamente

📌 Sistema para PUBLICAR Updates (solo **ADMIN**) - Actualización automática de **lastUpdate** en el negocio

Cuando un **ADMIN** publique un update:

1. Se crea el documento en: companies/{companyId}/businesses/{businessId}/updates
2. Se actualiza automáticamente en: companies/{companyId}/businesses/{businessId}

Los campos:

- lastUpdate
- (Opcional) status
- (Opcional) version

Todo automático.

♦ **BLOQUE 7** - Controlar permisos (Solo **ADMIN** publica)

- Mostrar botón “Publicar Update” solo si el usuario es **ADMIN**
- Solo **ADMIN** puede ver botón publicar
- **CLIENTE** y **SOPORTE** solo pueden leer
- Arquitectura limpia

Resultado a mostrar

- **ADMIN** ve botón
 - **CLIENTE** no lo ve
 - **SOPORTE** no lo ve
 - Seguridad lógica aplicada
-

FASE 5 — SISTEMA DE UPDATES (NOVEDADES)

Esta fase es para que el **admin** publique novedades del proyecto al cliente.

- Nueva versión subida
- Bug arreglado
- Nueva funcionalidad
- Mantenimiento

El cliente lo verá dentro de la app.

MEJORA 5.1 — SISTEMA COMPLETO DE UPDATES

- UpdatesActivity
- CreateUpdatesActivity
- UpdatesRepository
- UpdateAdapter
- UpdateModel

OBJETIVO

Completar el sistema de updates para que tenga:

- crear update
 - editar update
 - eliminar update
 - historial
 - ordenado por fecha
-

SISTEMA

El sistema final tendrá:

- Crear update
- Editar update
- Eliminar update
- Ordenar por fecha
- Mostrar historial

Después añadiremos:

- Notificaciones
- Badge de novedades

CAMPOS QUE DEBERÍA TENER UN UPDATE PROFESIONAL

Campo	Tipo Firebase	Variable Kotlin	Significado
id	string	String	ID Documento
title	string	String	Título
description	string	String	Descripción
version	string	String	Versión
type	string	String	Tipo Update
createdAt	number	Long	Fecha
createdBy	string	String	Admin/Usuario
status	string	String	Estado
isActive	boolean	Boolean	Visible
priority	number	Int	Importancia

Campo **type**

Valores recomendados

- feature
- bugfix
- maintenance
- security
- version

Campo **status**

Valores posibles

- draft
- published
- archived

Campo **priority**

Valores

- 1 = normal
- 2 = importante
- 3 = crítico

CreatedAt

Es la fecha que guarda cuando se creó el **update**

Es el **timestamp** o **number** (milisegundos)

En **Android** se usa los milisegundos desde 1970, llamado Unix **timestamp**

Qué significa el número 1710000000000: 9 Marzo 2024

Ese número representa una fecha real

No escribir esos números manualmente.

Android genera automáticamente con

- `System.currentTimeMillis()`
- “createdAt” to `System.currentTimeMillis()`

Cuando el admin cree un update, **Android** enviará los números largos y **Firestore** los guarda

EXPLICACIÓN DE LOS CAMPOS DE UN UPDATE PROFESIONAL

Campo	Tipo Firebase	Ejemplo	Significado
id	string	update123	ID Documento
title	string	Nuevo chat	Título
description	string	Añadido chat	Descripción
version	string	v1.4	Versión
type	string	feature	Tipo
createdAt	number	1719950234123	Fecha
createdBy	string	adminUID	Creador
status	string	published	Estado
isActive	boolean	true	Visible
priority	number	2	Importancia

MEJORA 5.2 — UPDATE FIJADO (PINNED)

Permitir que un update importante **siempre aparezca arriba**.

Por ejemplo: 📌 Versión 1.4 publicada

Esto lo usan sistemas como:

- **Discord** announcements
- **Slack** pinned messages

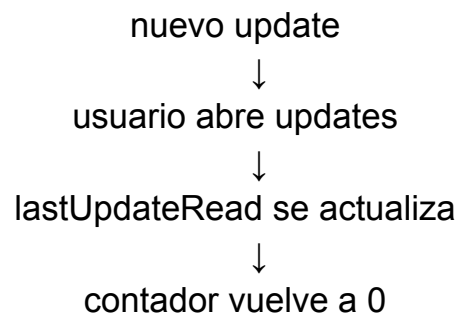
MEJORA 5.3 — CONTADOR DE UPDATES NO LEÍDOS

🎯 OBJETIVO

Saber qué updates ha leído cada usuario.

🎯 RESULTADO

El sistema funciona así:



6 FASE 6 - Chat en tiempo real (Firestore Realtime Listener)

♦ FASE 6 BASE

- Bloque 1–8 → UI + envío + burbujas
- Bloque 9 → Tiempo real
- Bloque 10 → Mostrar hora del mensaje

Estructura - Nivel **SaaS**

- Chat por negocio
- Tiempo real
- Mensajes con UID
- Distinción visual entre cliente y soporte
- Scroll automático
- Arquitectura limpia

Objetivo: Crear un chat por negocio donde:

- Cada negocio tenga su propio chat
- **CLIENTE** pueda escribir
- **SOPORTE / ADMIN** puedan responder
- Mensajes en tiempo real
- Diferenciación visual por usuario
- Scroll automático
- Arquitectura limpia (Repository + ViewModel + UI)

◆ BLOQUE 1 - Estructura en Firebase

companies/{companyId}/businesses/{businessId}/chat

Cada mensaje tendrá

```
{
  "message": "Hola, ¿cómo va el desarrollo?",
  "senderId": "UID_DEL_USUARIO",
  "senderRole": "CLIENT",
  "createdAt": timestamp
}
```

Un mensaje = un documento.

Si hay 100 mensajes → 100 documentos.

No se guardan varios senderId en uno.

No se guardan varios mensajes en uno.

Cada mensaje es independiente.

◆ BLOQUE 2 - ChatModel

Documento por mensaje, es decir, cada mensaje tendrá:

- id
- message
- senderId
- senderRole
- createdAt

◆ BLOQUE 3 - ChatRepository

- Enviar mensaje
- Escuchar mensajes en tiempo real (addSnapshotListener)

La App pueda leer los mensajes del chat desde Firestore/Firebase

◆ BLOQUE 4 - ChatViewModel

- Estado observable
- Control de errores
- Control de loading

ChatViewModel sólo lee los mensajes, no puede enviar mensajes.

🎯 OBJETIVO

- Llamar a **ChatRepository**
- Obtener los mensajes
- Exponerlos como **LiveData**
- Que la UI pueda observarlos

◆ BLOQUE 5 - Layout

- **ChatActivity**
- item_message.xml
- **RecyclerView**
- Caja de escribir mensaje
- Botón enviar

🎯 ESTADO ACTUAL

Ahora ya puedes:

- Abrir **ChatActivity**
- Ver mensajes cargados desde **Firebase**
- Mostrar texto simple en pantalla

◆ BLOQUE 6 - Adapter avanzado

- Mensaje a la derecha si es mío
- Mensaje a la izquierda si es del otro
- Diferente fondo

✍️ Enviar mensaje a Firebase (WRITE)

- Escribir un mensaje desde la app
- Guardarlo en Firestore
- Que aparezca en el chat

ChatRepository

- Crea un **Map**
- Añade documento nuevo automático
- Guarda fecha actual
- Usa **.await()** para que sea suspend

🎯 RESULTADO

Ahora puedes:

- Escribir mensaje
- Pulsar enviar
- Se guarda en **Firestore**
- Se vuelve a cargar la lista

◆ BLOQUE 7 - Seguridad

- Solo usuarios autenticados
- Solo usuarios del negocio pueden escribir
- Roles controlados

Estado actual del chat:

- Carga mensajes
- Envía mensajes
- Refresca la lista
- Pero **no hace scroll automático**
- Y puede no bajar al último mensaje

🎯 OBJETIVO

Cuando:

- Se cargan mensajes
- Se envía un mensaje

👉 El **RecyclerView** baje automáticamente al último mensaje.

Esta función: `scrollToPosition(messages.size - 1)` hace que cuando lleguen mensajes nuevos: Hace scroll al último elemento.

🧠 Qué hace `stackFromEnd`

Hace que el **RecyclerView**:

- Empiece desde abajo
- Como WhatsApp
- Y mantenga el foco en el último mensaje

🎯 ESTADO ACTUAL DEL CHAT

- ✓ Leer mensajes
- ✓ Enviar mensajes
- ✓ Scroll automático
- ✓ Orden por fecha
- ✓ Arquitectura limpia (Repository + ViewModel)

◆ BLOQUE 8 - Mejoras profesionales

- Scroll automático
- Fecha bonita
- Estado “visto”
- Bloqueo si isActive = false

🧠 Diferenciar burbujas (derecha / izquierda)

🎯 Objetivo:

- Mis mensajes → derecha
- Mensajes del otro → izquierda
- Diferente color
- Chat estilo WhatsApp profesional

🧠 CONCEPTO IMPORTANTE

Para saber si un mensaje es mío o no, comparamos: senderId == currentUser.uid → Si coinciden → es mío.

🎯 RESULTADO

- Tus mensajes → derecha, verde
- Otros mensajes → izquierda, gris
- Scroll automático funcionando
- Chat empieza a parecer real

🎯 ESTADO ACTUAL DEL CHAT

Flujo chat - Envía mensaje - Luego vuelve a cargar con **loadMessages()** Pero no escucha cambios en vivo

- Envío funcional
- Lectura funcional
- Scroll automático
- Burbujas derecha / izquierda
- Arquitectura limpia

◆ BLOQUE 9 - Tiempo real con `addSnapshotListener`

- Si alguien escribe
- Si tú escribes
- Si otro admin escribe

El mensaje aparece automáticamente sin recargar.

🧠 ¿Qué hace esto?

- Se queda escuchando Firestore
- Cada vez que cambia algo
- Devuelve la lista actualizada

Sin `await`.

Sin recarga manual.

🎯 RESULTADO

Funcionamiento del chat:

- Es 100% en tiempo real
- No necesita recargar
- Funciona como WhatsApp
- Se sincroniza entre usuarios

◆ BLOQUE 10 - Mostra hora del mensaje

Ahora podemos:

- Mostrar hora debajo del mensaje
- Mostrar nombre del emisor
- Indicador de leído
- Notificaciones

🎯 RESULTADO

- Tiempo real
- Burbujas izquierda/derecha
- Hora debajo del mensaje
- Scroll automático
- Arquitectura limpia

● ESTADO ACTUAL DE LA FASE 6

Implementaremos exactamente esto, en orden estratégico:

- Hora
- Nombre remitente
- Estados mensaje (read/delivered)
- Indicador escribiendo
- Animaciones
- DiffUtil
- Paginación

Chat actual:

- Envío de mensajes
- Lectura
- Tiempo real con **addSnapshotListener**
- Scroll automático
- Burbujas derecha/izquierda
- Arquitectura limpia (**Repository** + **ViewModel**)

FASE 6 - SISTEMA DE NOTIFICACIONES

Eventos que dispararán notificación:

- nuevo mensaje de chat
- nuevo update publicado
- respuesta del soporte

OBJETIVO

Que el usuario reciba una notificación cuando:

- admin publica update
- admin responde chat

ARQUITECTURA

El sistema funciona así:



RESULTADO

App del chat:

- Notificaciones push
- Sistema profesional
- Avisos de soporte
- Avisos de Update

MEJORAS PROFESIONALES FASE 6 - PARTE 1

1) Mostrar hora del mensaje

- Usando createdAt.
- Impacto en rendimiento: NULO
- Nivel profesional: **Alto**
- Recomendación: Sí o sí

Usando createdAt.

Impacto en rendimiento: NULO

Nivel profesional: **Alto**

Recomendación: Sí o sí

2) Mostrar nombre del remitente (solo si hay varios admins)

Si una empresa tiene 3 desarrolladores:

- Carlos:
- Mensaje...

Solo mostrar nombre cuando:

- No sea tu mensaje
- O haya varios admin

Impacto: **Bajo**

Recomendación: Sí

3) Estados del mensaje (Enviado / Entregado / Leído)

- ✓ Enviado
- ✓✓ Entregado
- ✓✓ Azul Leído

Esto requiere:

- Campo **isRead**
- Campo **deliveredAt**

Impacto: Bajo si se diseña bien

Complejidad: **Media**

Recomendación: Para versión avanzada

④ Indicador “Escribiendo...”

Cuando alguien está escribiendo

Requiere:

- Campo temporal en **Firestore**
- Borrado automático

Impacto: Bajo

Nivel UX: **Muy alto**

Recomendación: Sí, pero después de cerrar fase base

⑤ Animaciones suaves

- Animación al aparecer mensaje
- Fade in
- Slide

Impacto: **Muy bajo**

Recomendación: Sí

⑥ Optimización real

`notifyDataSetChanged()` - **DiffUtil**

Esto hace que:

- No redibuje toda la lista
- Solo actualice lo nuevo

Impacto rendimiento: Muy positivo

Recomendación: Sí obligatorio en versión profesional

⑦ Paginación (para chats largos)

Cargar sólo los últimos 30 mensajes.

Impacto: Muy positivo

Recomendación: Cuando haya uso real

① Añadir hora del mensaje (Bloque 10)

② Añadir nombre del remitente

③ Implementar DiffUtil (optimización real)

④ Indicador escribiendo

⑤ Estado leído

FASE 6 PRO - FASE 1

 FASE 6 PRO - Mejora 1 |  Mostrar nombre del remitente

RESULTADO

Ahora el chat:

- Si hay 3 desarrolladores → se ve quién escribe
- El cliente sabe quién responde
- Profesional empresarial
- Sin afectar rendimiento
- Cada mensaje guarda el nombre del emisor
- Se muestra solo si no es tu mensaje
- No afecta rendimiento
- Arquitectura correcta

Seguridad real

- Reglas Firestore por miembro
- Admin-only delete

Sistema unread automático

- Incremento automático
- Reset al abrir

Notificaciones push respetando mute

Parcial — ahora mismo:

- Token guardado ✓
- Servicio FCM creado ✓
- Envío condicional real ✗ (falta backend real o lógica completa)

Arquitectura profesional

- UseCase añadido
- Separación básica domain
- Clean Architecture completa aún no

🚀 FASE 6 PRO - Mejora 2 | ✅ Estados del mensaje (Enviado / Entregado / Leído)

🎯 OBJETIVO

En tus mensajes (solo los tuyos) queremos mostrar:

- 🕒 Enviado
- ✓ Entregado
- ✓✓ Leído

Flujo

1 Cuando envías → se crea con:

- **isDelivered** = false
- **isRead** = false

2 Cuando el receptor abre el chat:

- Todos los mensajes que no son suyos → **isDelivered** = true
- Y luego **isRead** = true

🎯 RESULTADO

Ahora el chat:

- Marca mensajes como leídos automáticamente
- Muestra estado debajo del mensaje propio
- No afecta rendimiento
- Arquitectura limpia

🚀 FASE 6 PRO - Mejora 3 | ✍️ Indicador “Escribiendo...”

🎯 RESULTADO

- Si alguien escribe → aparece indicador
- Se sincroniza en tiempo real
- Desaparece automáticamente
- No afecta rendimiento
- No crea mensajes falsos
- Arquitectura limpia y creada en Firebase

FASE 6 PRO - Mejora 4 | DIFFUTIL

RESULTADO

- DiffUtil real
- ListAdapter profesional
- Animaciones suaves
- Sin notifyDataSetChanged
- Sin redibujar toda la lista
- Mejor rendimiento en chats largos

FASE 6 PRO - Mejora 5 | ESTADOS DEL MENSAJE (sent → read)

- 1) Firebase
- 2) Android (Model → Repository → Activity → Adapter)
- 3) Commit en GitHub

PASO 1 — FIREBASE (MUY IMPORTANTE)

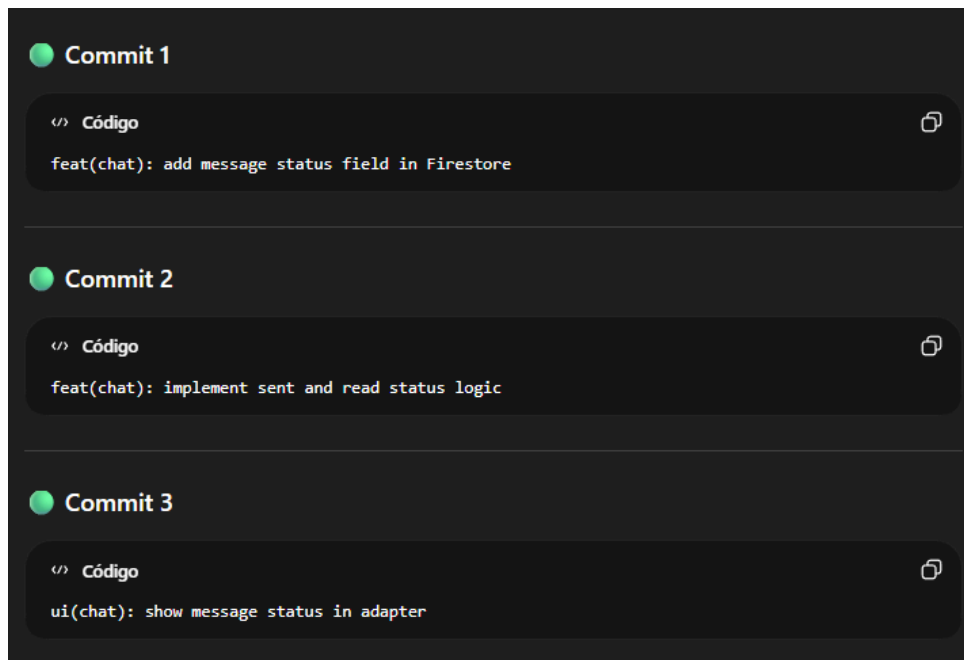
En cada mensaje dentro de:

 Código

```
companies
  NGDStudios
    businesses
      Restaurante_madrid
        chat
          (mensajeId)
```

Cada mensaje debe tener:

Campo	Tipo
id	string
message	string
senderId	string
senderName	string
timestamp	timestamp
status	string



🚀 FASE 6 PRO - Mejora 6 | Hora formateada profesional

Vamos a hacerlo tipo WhatsApp:

- Hoy → 14:32
- Ayer → Ayer 14:32
- Otro día → 12 Ene 14:32

🚀 RESUMEN

- Hora profesional
- Nombre correcto
- Estado real automático
- Optimización con DiffUtil

🔥 AHORA TIENES:

- Paginación real
- Scroll infinito
- Separadores de fecha
- Indicador nuevo mensaje
- Animaciones suaves

2 FIJAR MENSAJE ARRIBA

Concepto:

Mensaje con campo:

```
</> Código  
isPinned: true
```

Se muestra arriba del chat.

¿Quieres que el mensaje fijado se muestre:

- A) Como barra fija arriba del todo
- B) Como mensaje flotante arriba
- C) Como sección aparte tipo Telegram

Respóndeme A, B o C y lo implementamos profesionalmente.

3 INDICADOR MENSAJES NO LÉIDOS REAL

Esto requiere:

- Campo unreadCount por usuario
- Incrementarlo en Firebase
- Resetearlo al abrir chat

Esto ya es arquitectura multiusuario real.

Lo implementamos después del fijado.

4 ANIMACIONES SUAVES REALES (DiffUtil real)

Después sustituimos submitMessages por:

- ListAdapter
- DiffUtil.ItemCallback



Eso elimina notifyDataSetChanged completamente.

🎯 OBJETIVO

Convertir tu chat en:

- 📁 Sistema tipo Telegram
- 🧩 Canales / conversaciones estructuradas
- 🔔 Indicador real de no leídos por usuario
- 🧠 Persistente (NO se pierde)
- 🚀 Animaciones suaves reales con ListAdapter + DiffUtil

🔵 ¿CÓMO FUNCIONA unreadCount?

Cuando alguien envía mensaje:

- Se incrementa unreadCount de todos menos el sender.
- No se borra historial.
- Solo se incrementa el número.

Cuando el usuario entra al canal:

- Se pone su contador a 0.
- No se borra nada.
- Solo se marca como leído.

🎯 RESULTADO

- Lista tipo Telegram
- Badge rojo real conectado a unreadCount
- Animaciones suaves reales con ListAdapter
- DiffUtil profesional
- Arquitectura escalable

🔵 Mejora 7 — Marcar canal como leído al abrirlo

🎯 Objetivo

Cuando el usuario toque un canal:

- Se abre ChatActivity
- Se pone unreadCount[currentUserId] = 0
- No se borra nada
- Solo se marca leído

🔵 Mejora 8 - Llamar a markChannelAsRead desde ChatActivity

🔵 Mejora 9 - Ordenar canales por último mensaje

🔵 Mejora 10 - Preparar fijar canal arriba

🎯 RESULTADO

- Reset automático al abrir canal
- Orden tipo Telegram
- Badge real persistente
- Canales fijables arriba
- Arquitectura escalable

1 SWIPE PARA FIJAR CANAL

Objetivo

- Deslizar a la derecha → pinned = true
- Deslizar a la izquierda → pinned = false

2 CONTADOR GLOBAL TOTAL DE NO LEÍDOS

3 ANIMACIÓN PROFESIONAL AL RECIBIR NUEVO CANAL

4 INDICADOR FLOTANTE “NUEVO MENSAJE”

RESULTADO FINAL

- Swipe para fijar canal
- Orden por pinned + último mensaje
- Contador global real
- Animaciones suaves profesionales
- Indicador flotante nuevo mensaje
- Arquitectura estilo Telegram

BLOQUE 1 — Ocultar canales archivados en UI

- Campo isArchived en canal
- Filtrado en ViewModel
- Actualización en UI
- Botón para archivar
- Commit

OBJETIVO DEL BLOQUE 1

- Cada canal tendrá un campo isArchived en Firebase
- Podrás archivar / desarchivar un canal
- Los canales archivados NO aparecerán en la lista principal
- El sistema seguirá guardando el canal en Firestore
- Tendremos commit final profesional

RESULTADO FINAL DEL BLOQUE 1

Cuando archives un canal:

- Se actualiza en Firestore
- isArchived pasa a true
- Desaparece automáticamente de la lista
- No se borra el canal
- Los mensajes siguen intactos

BLOQUE 2 — Control real de permisos + Mute + Archivo

- Campo mutedUsers en canal
- Comprobación antes de enviar notificación
- Lógica condicional
- Commit

OBJETIVO DEL BLOQUE 2

- Bloquear escritura si no es admin (en Firestore REAL)
- Sistema real de muted por usuario
- Evitar notificaciones si muted = true
- Ocultar archivados correctamente
- Reglas Firestore profesionales por roles

Resultado Reglas Firebase

- Cliente NO puede borrar canales
- Cliente NO puede modificar canales
- Solo admin puede borrar mensajes
- Seguridad REAL de producción

BLOQUE 3 — Bloquear escritura si no es admin

- Comprobación de rol en UI
- Desactivar botón enviar
- Reglas Firestore para write
- Commit

Resultado

- Unread automático
- Estilo WhatsApp
- Escalable

BLOQUE 4 — Reglas reales Firestore

- Solo miembros pueden leer
- Solo admin puede borrar canal
- Solo miembros pueden escribir mensajes
- Deploy reglas
- Commit

OBJETIVO

- Solo miembros pueden leer canal
- Solo admin puede borrar canal
- Solo miembros pueden escribir mensajes

BLOQUE 5 — Sistema automático unreadCount

- Incremento automático al enviar mensaje
- Reset al abrir canal
- Actualización Map unreadCount
- Commit

OBJETIVO

- Incrementa unreadCount al enviar mensaje
- Reset cuando usuario abre canal

BLOQUE 6 — Notificaciones push respetando mute

- Firebase Cloud Messaging
- Comprobación mutedUsers
- Envío condicional
- Commit

¿QUÉ ES EL FCM TOKEN?

- Cuando instalas la app en un móvil:
- **Firestore** genera un **identificador único para ese dispositivo**.
- Ese identificador se llama:
 - 👉 **FCM Token**
 - Es como la dirección del móvil para enviar notificaciones
- Si no guardas ese **token** en **Firestore**
- NO puedes enviar notificaciones a ese usuario

¿CUÁNDO SE EJECUTA?

Cada vez que el usuario entra en **Dashboard**.
Firestore mantiene el **token** automáticamente.

¿CÓMO SE USA ESE TOKEN?

Más adelante, cuando tengamos backend (**Cloud Function**):

1. Leemos **members** del canal
2. Leemos cada user/{uid}/**fcmToken**
3. Enviamos notificación a esos **tokens**

 **¿Cómo ver el token que se genera?**

En `DashboardActivity`, temporalmente añade esto:

```
</> Kotlin

FirebaseMessaging.getInstance().token.addOnCompleteListener { task ->
    if (!task.isSuccessful) return@addOnCompleteListener

    val token = task.result
    println("FCM TOKEN: $token")
}
```

Ahora:

1. Ejecuta la app en el móvil o emulador
2. Abre Logcat
3. Busca: `FCM TOKEN`

Verás algo como:

```
</> Código

FCM TOKEN: eR9sdfj2394kjsdf9234...
```

Ese es el token REAL del dispositivo.

BLOQUE 7 — Arquitectura profesional final

Incluye:

- Separación domain layer
- UseCases
- Clean Architecture
- Modularización futura
- Commit

● ESTADO ACTUAL

- ✓ Arquitectura Activity → **ViewModel** → **Repository**
- ✓ Coroutines bien usadas
- ✓ **unreadCount** automático
- ✓ Reset unread al abrir
- ✓ FCM token guardado en **users/{uid}**
- ✓ **ChannelViewModel** con **UseCase**
- ✓ Firestore rules básicas
- ✓ Cast seguro en **Map**
- ✓ Eliminado acceso directo a Repository desde **Activity**

1 FCM real con envío condicional

- Guardas token ✓
- Recibes notificación ✓
- Pero NO estás enviando push desde servidor

Necesitarías:

- Cloud Function
- O servidor Node
- Leer members del canal
- Filtrar mutedUsers
- Enviar push solo a no muteados

2 Validación unread server-side

Ahora unread se incrementa desde cliente.

Un usuario malicioso podría manipularlo.

Versión PRO real sería:

- Incrementar unread desde Cloud Function onCreate message

Actualmente es cliente-side.

3 Clean Architecture REAL

UI → ViewModel → UseCase → Repository → Firebase

Pero aún no tienes:

- Interfaces en domain
- Repository abstraído
- Inyección de dependencias
- Separación de módulos

6 FASE 6 PRO - FASE 2 - PLAN SPARK FIREBASE

● MEJORA 1 — UnreadCount 100% server-side (Cloud Function)

◆ Qué incluye

- Eliminar incremento unread desde Android
 - Crear Cloud Function **onMessageCreate**
 - Incrementar unread automáticamente
 - Evitar manipulación cliente
 - Deploy a Firebase
-

● MEJORA 1 — UNREAD AUTOMÁTICO SERVER-SIDE

🎯 Objetivo

Cuando alguien envíe un mensaje:

1. Firestore detecta que se creó un documento
2. Cloud Function se activa automáticamente
3. Incrementa unreadCount de todos menos el sender
4. El cliente ya NO incrementa nada

🎯 Resultado

- Nadie puede manipular unread
- Es automático
- Es seguro

PASO 1 — Instalar Node correctamente

1. Ve a <https://nodejs.org>
2. Descarga LTS
3. Instálalo
4. Reinicia el ordenador

Luego abre PowerShell o CMD nuevo.

- `node -v` - Debe mostrar versión

Luego ejecuta: `npm install -g firebase-tools`

- Después `firebase --version`

PASO 2 — Inicializar Functions

En la raíz del proyecto (donde está `app/`):

`firebase login`

`firebase init functions`

Selecciona:

- Use existing project
- JavaScript
- No ESLint
- Yes install dependencies

Se crea carpeta: `functions/`

PASO 3 — Crear función unread

Abre: `functions/index.js`

PASO 4 — Deploy

`firebase deploy --only functions`

PASO 5 — ELIMINAR `incrementUnread` DEL ANDROID

En **ChatRepository** elimina: `incrementUnread(...)`

RESULTADO FINAL MEJORA 1

- unreadCount ahora es server-side
 - Cliente no puede manipular
 - Seguridad mejorada
 - Sistema automático real
-

MEJORA 2 — Notificaciones Push reales desde servidor

- Cloud Function:
 - Lee miembros del canal
 - Lee mutedUsers
 - Filtra usuarios muteados
 - Lee sus fcmToken
 - Envía notificación sólo a los correctos
 - Uso de Firebase Admin SDK
 - Deploy
-

NOTIFICACIONES PUSH - SERVIDOR CON MUTE

OBJETIVO

Cuando alguien envía un mensaje:

1. Se activa Cloud Function
2. Lee miembros del canal
3. Lee mutedUsers
4. Filtra usuarios muteados
5. Lee sus **fcmToken**
6. Envía notificación solo a los que:
 - No son el sender
 - No están muteados
 - Tienen token válido

Resultado

- Push real
 - Respeta mute
 - No depende del cliente
 - Profesional
-

PASO 1 — Verificar estructura Firestore

```
</> Código 

companies/{companyId}/businesses/{businessId}

{
  members: {
    uid1: { role: "admin" },
    uid2: { role: "member" }
  },
  mutedUsers: {
    uid1: false,
    uid2: true
  }
}
```

Y los tokens guardados en:

```
</> Código 

users/{uid}
  fcmToken: "token..."
```

PASO 2 — Modificar Cloud Function

Vamos a modificar la función:

- Incremente unread
- Envíe notificaciones

PASO 3 — Deploy de la función

En terminal - `firebase deploy --only functions`

RESULTADO FINAL MEJORA 2

- Push 100% server-side
 - Respeta muteUsers
 - No depende del cliente
 - No se puede manipular
 - unread + push integrados
 - Arquitectura backend real
-

MEJORA 3 — Validación fuerte en Firestore Rules

♦ Qué incluye

- Impedir que cliente modifique unreadCount
 - Impedir que cliente modifique mutedUsers de otros
 - Validar miembros correctamente
 - Reglas finales de producción
-

MEJORA 3 — REGLAS FIRESTORE REFORZADAS

¿Qué bloqueamos exactamente?

- Nadie puede modificar members
- Nadie puede modificar unreadCount manualmente
- Solo admin puede borrar canal
- Solo miembros escriben mensajes
- Cada usuario solo puede modificar su propio documento

RESULTADO MEJORA 3

- Seguridad fuerte incluso sin Cloud Functions
- unread no manipulable
- mutedUsers protegido
- Sistema listo para producción pequeña

MEJORA 4 — Clean Architecture real mínima profesional

- Interface en domain
 - Repository implementation en data
 - UseCase desacoplado
 - ViewModel limpio
 - Preparación para modularización futura
-

MEJORA 4 — ARQUITECTURA LIMPIA ESTABLE Y PREPARADA PARA ESCALAR

¿QUÉ INCLUYE ESTA MEJORA?

1. Separar responsabilidades correctamente
 2. Preparar el proyecto para Clean Architecture futura
 3. Desacoplar Repository del ViewModel mediante interfaz
 4. Dejar el sistema listo para:
 - Modularización futura
 - Inyección de dependencias
 - Migración a Blaze sin romper nada
-

¿QUÉ INCLUYE?

1. Asegurar que ninguna Activity accede directamente a Firebase
2. Asegurar que ninguna Activity crea repositorios directamente
3. Asegurar que toda la lógica pasa por ViewModel
4. Centralizar acceso a datos en repositories
5. Dejar el proyecto listo para escalar a Blaze en el futuro

ESTADO FINAL FASE 6

- Unread automático
- Sistema mute
- Reglas Firestore seguras
- Arquitectura limpia básica
- Preparado para migración a Blaze

PLAN DE EJECUCIÓN REAL

BLOQUE 1 — UnreadCount 100% server-side

Qué incluye:

1. Crear campo **unreadCount** dentro de:

companies/{companyId}/businesses/{businessId}/chatStatus/{uid}

2. Cada vez que se envía mensaje:
 - Incrementar unreadCount de los demás usuarios
 - No incrementar el del emisor
3. Al abrir el chat:
 - Poner unreadCount = 0

PASO 1 - Cuando alguien mande mensaje:

- Buscar todos los documentos en **chatStatus**
- Para cada usuario:
 - Si NO es el emisor → incrementar unreadCount +1
 - Si es el emisor → no tocar

Cuando alguien abre chat:

- Poner su unreadCount en 0

PASO 2 - Incrementar unreadCount al enviar mensaje

● BLOQUE 2 — Clean Architecture mínima profesional

Separaremos tu proyecto en:

```
data/  
  repository/  
  datasource/  
domain/  
  model/  
  repository/  
  usecase/  
ui/
```

Qué haremos:

1. DESARROLLO DE APLICACIONES MÓVILES CON IACrear interfaces en **domain**
2. Mover implementaciones a **data**
3. Crear UseCases
4. ViewModel solo hablará con UseCases
5. Eliminar dependencias directas de Firebase desde ViewModel

● Mejora Mostrar datos reales del Business en AppDetailActivity

Vamos a hacer que cargue:

- Nombre
- Estado
- Progress
- Versión
- Soporte
- Última actualización

1 Qué incluye esta mejora

- Cargar Business desde Firestore
 - Mostrar datos reales en AppDetailActivity
 - Usar ViewModel + Repository (arquitectura correcta)
 - Preparar base para:
 - Chat
 - Updates
 - IA Support
-

Sistema de Updates por negocio (Changelog)

Añadimos una sección Updates para cada negocio. Cada negocio tendrá su historial de mejoras

Esto permitirá:

- Mostrar avances al cliente
- Transparencia en desarrollo
- Historial profesional de cambios

1 Chat completo

Sistema de chat completo:

- Enviar mensaje
- Guardar mensaje en Firebase
- Escuchar mensajes en tiempo real
- Mostrar mensajes en RecyclerView
- Tenga el ID correcto

Documento mensaje:

- text: "Hola necesito soporte"
- senderId: "uid123"
- timestamp: 1712000000

Resultado | Parte 1

- Entrás al chat
- Escribes mensaje
- Se guarda en Firebase
- Aparece en colección messages

Qué incluye

Añadimos al chat:

- Contador de mensajes no leídos
- Marcar chat como leído al abrirlo
- Actualizar **unreadCount** en Firestore
- Preparado para notificaciones FCM después

Resultado final | Parte 2

- Envío de mensajes
- Responder mensajes
- Separador por fecha
- Scroll automático
- Marcar chat como leído

Y **Firestore** guardará: unreadCount_user1 = 0 - unreadCount_user2 = 3

2 Crear Updates desde panel Admin

Panel admin para:

- Panel Admin
- Crear y Publicar update
- Guardar update en Firebase
- Mostrar en **UpdateActivity**
- Que cliente lo ve
- Opciona: Notificación **push**

Documento Update:

- title: "Nueva versión disponible"
- description: "Se ha corregido error login"
- timestamp: 171200000

3 Dashboard preparado para múltiples empresas

MEJORA 7 - CONTADOR DE MENSAJES NO LEÍDOS

Objetivo

Sistema profesional de chat:

- Contador de mensajes no leídos
- Reset al abrir chat
- Badge en dashboard
- Incremento automático
- Arquitectura tipo Slack

RESULTADO

El chat ahora funciona como apps reales:

- Mensajes incrementan el contador
- Contador depende de quién envía
- Al abrir chat se resetea
- Dashboard puede mostrar badges

BLOQUE 3 — Preparar estructura para IA

Antes de meter IA, necesitamos:

1. Crear módulo **ai/**
2. Crear interfaz **AiRepository**
3. Crear UseCase **SendMessageToAiUseCase**
4. Preparar canal "IA_SUPPORT"

No usaremos aún OpenAI real.

Solo dejaremos arquitectura preparada.

BLOQUE 4 — FASE 7 (Integración IA real)

- Llamadas HTTP
- Seguridad
- API Key segura
- Respuestas automáticas
- Guardado en Firestore



- Mejorar formato
- Crear y Convertir a RecyclerView
- Refactorizar a MVVM y convertirlo de manera completa
- Refactorizar repositorios
- Añadir sealed classes de estado

Si quieres, el siguiente paso puede ser:

Definir qué va a hacer exactamente UpdatesActivity (¿lista de novedades por app?).

Diseñar su modelo y repo.

Crear UpdatesViewModel + UpdatesUiState + RecyclerView, igual que el Dashboard.

GitHub Commit & Push

- Initial Android project setup
- Add Chat and Updates activities with base layouts
- Integrate Firebase Authentication (email/password)
- Phase 3 - Apps linked to user and displayed@t
- Phase 4 - Updates system implemented
- Phase 5 - Changes Cursor - RecyclerView - AppAdapter - tcApps - MVVM + estados
- Phase 6 - Add user roles and passwords
- Phase 7 - Firebase: Created ADMIN and CLIENT users with roles
- Phase 8 - Dashboard: Load user info (name, role, company) from Firestorees
- Phase 9: Implement multi-company structure and role filtering

GitHub Commit & Push - **UPDATES**

FASE 4 - Bloque 1: ESTRUCTURA PROFESIONAL DE business EN FIRESTORE

FASE 4 - Bloque 2: Actualizado BusinessModel con campos profesionales de estado

FASE 4 - Bloque 3: Ampliado AppModel con campos de estado y progreso

FASE 4 - Bloque 3: Dashboard ahora pasa todos los datos completos del Business al AppModel

FASE 4 - Bloque 4: Nuevo diseño profesional del item del Dashboard con progreso y soporte

FASE 4 - Bloque 4: Adapter ahora muestra progreso, versión y tipo de soporte

FASE 4 - Bloque 5: Añadidos colores de estado en colors.xml

FASE 4 - Bloque 5: Estado ahora cambia de color dinámicamente

FASE 4 - Bloque 5: ProgressBar ahora cambia de color según estado

FASE 4 - Bloque 5: Añadidos iconos dinámicos por estado

FASE 4 - Bloque 5: Badge visual dinámico profesional para estado

FASE 4 - Bloque 6: Filtrado automático de negocios inactivos

FASE 4 - Bloque 6: Protección de progreso entre 0 y 100

FASE 4 - Bloque 6: Preparada función para actualización automática de lastUpdate

FASE 5 - Bloque 1: Creada estructura updates en Firestore por business

FASE 5 - Bloque 2: Creado UpdateModel alineado con Firestore

FASE 5 - Bloque 3: Creado UpdatesRepository con lectura y creación

FASE 5 - Bloque 4: Creado UpdatesUiState

FASE 5 - Bloque 4: Creado UpdatesViewModel con carga de datos

FASE 5 - Mejora: Updates ordenados por fecha descendente

FASE 5 - Bloque 5: Creado layout UpdatesActivity

FASE 5 - Bloque 5: Creado UpdateAdapter

FASE 5 - Bloque 5: UpdatesActivity conectada al ViewModel

FASE 5 - Bloque 6: Añadida función publishUpdate

FASE 5 - Bloque 6: Creado layout PublishUpdateActivity

FASE 5 - Bloque 6 (Fix): PublishUpdateActivity funcional - Constructor correcto en UpdatesRepository

FASE 5 - Bloque 7: Control de permisos para publicar updates

FASE 6 - Bloque 2: Creado ChatMessageModel

FASE 6 - Bloque 3: Leer mensajes de Firebase

FASE 6 - Bloque 4: Conexión y llamada con Firebase. ViewModel solo lee mensajes

FASE 6 - Bloque 5: Creada ChatActivity, Adapter y Layout base del chat

FASE 6 - Bloque 6: Envío de mensajes a Firebase implementado

FASE 6 - Bloque 7: Scroll automático implementado en Chat

FASE 6 - Bloque 8: Chat visual con burbujas diferenciadas y scroll automático y Diferenciación visual de mensajes implementada

FASE 6 - Bloque 9: Chat en tiempo real implementado con addSnapshotListener

FASE 6 - Bloque 10: Hora de mensaje implementada

FASE 6 PRO - Mejora 1: Nombre del remitente implementado correctamente

FASE 6 PRO - Mejora 2: Estados de mensaje implementados (Enviado / Entregado / Leído)

FASE 6 PRO - Mejora 3: Indicador escribiendo implementado

FASE 6 PRO - (Mejora 4): Implementado DiffUtil y animaciones suaves en Chat

FASE 6 PRO - Mejora 5 | ESTADOS DEL MENSAJE (sent → read)- add message status field in Firestore - add message status field in Firestore - implement sent and read status logic

FASE 6 PRO - Mejora 6 y 7 - refactor: fix ChatActivity and ChatAdapter structure - feature: implement professional pagination with limit and startAfter - feature: add automatic read status and professional timestamp formatting

FASE 6 PRO - Mejora 8 - refactor: stabilize chat activity and pagination logic - feature: add date separators like WhatsApp - feature: add new message indicator and smooth animations

FASE 6 PRO - Mejora 9 - feature(chat): add date separators, message highlight and restructure chat architecture

FASE 6 PRO - Mejora 10 - feature(chat): implement swipe reply with referenced message support

FASE 6 PRO - Mejora 11 - feature(chat): reply visual chat

FASE 6 PRO - Mejora 12 - git commit -m "feature(channel): add channel model aligned with firestore structure"

FASE 6 PRO - Mejora 13 - git add ChatRepository.kt

git commit -m "feature(channel): implement channel listener from firestore"

FASE 6 PRO - Mejora 14 - git commit -m "feature(channel): add channel viewmodel"

FASE 6 PRO - Mejora 15 - git commit -m "ui(channel): add channel item layout with unread badge"

FASE 6 PRO - Mejora 16 - git commit -m "feature(channel): implement ListAdapter with DiffUtil and unread badge"

FASE 6 PRO - Mejora 17 - git add ChannelActivity.kt activity_channel.xml

git commit -m "feature(channel): implement channel list screen with realtime unread badge"

FASE 6 PRO - git add ChatRepository.kt

git commit -m "feature(unread): implement markChannelAsRead when opening channel"

FASE 6 PRO - git commit -m "feature(unread): reset unreadCount for current user when channel opened"

FASE 6 PRO - git add ChatRepository.kt

git commit -m "feature(channel): update lastMessageTimestamp when sending message"

FASE 6 PRO - git add ChatRepository.kt

git commit -m "feature(channel): order channels by last message timestamp descending"

FASE 6 PRO - git add ChatRepository.kt

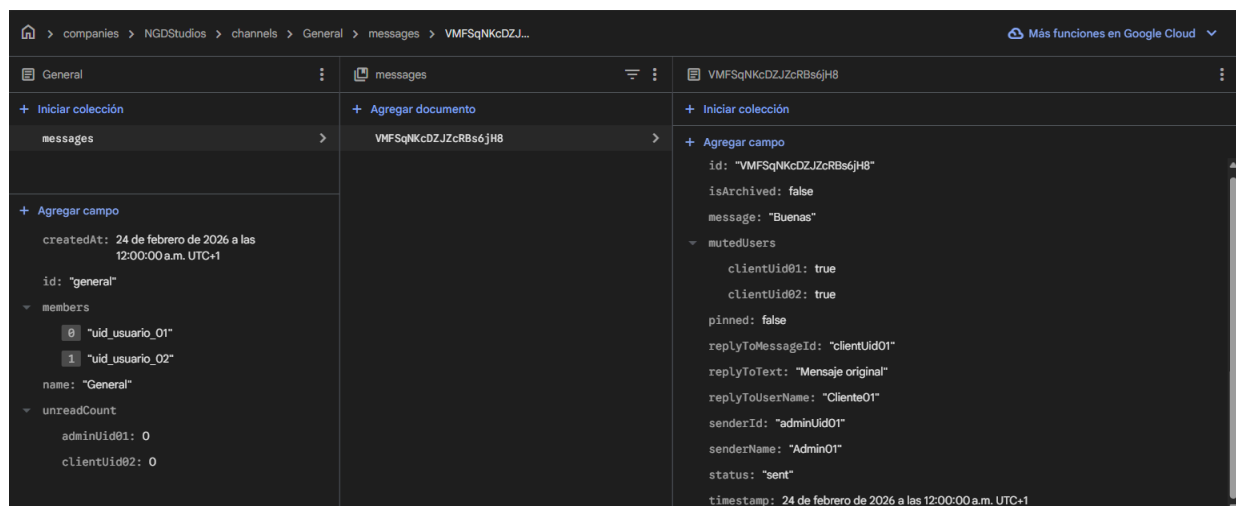
git commit -m "feature(channel): support pinned channels ordering"

git commit -m "feature(channel): add setChannelPinned support"

git commit -m "feature(channel): implement swipe to pin/unpin channel"
 git commit -m "feature(unread): add global unread counter"
 git commit -m "ui(channel): add layout animation for smooth channel appearance"
 git commit -m "feature(chat): add floating new message indicator"
 FASE 6 PRO - git adapter - submitMessages functional - Reply - Layout clear
 -m "feature(chat): implement telegram-style reply system with preview"
 feat: add reply structure to firestore messages
 feat: add reply fields to ChatMessageModel

FASE 6 PRO git commit -m "Implement channel system with roles, mute, archive and private channel creation - Added ChannelMember model - Updated ChannelModel to use members as Map<String, ChannelMember> - Implemented ChannelRepository - Implemented ChannelViewModel - Added muteChannel and archiveChannel logic - Added private channel creation - Updated Firestore structure for channels"

FASE 6 PRO git commit -m "Implemented channel system base structure - ChannelMember model - ChannelModel with members Map - ChannelRepository - ChannelViewModel - Private channel creation - Mute and archive fields in Firestore"



FASE 6 PRO git commit -m Solutions createPrivateChannel

BLOQUE 1 - git commit -m "Implemented channel archive system - Added isArchived field in Firestore - Updated ChannelModel - Added archiveChannel in Repository - Added archiveChannel in ViewModel - Filtered archived channels from UI"
 BLOQUE 2 y 3 - "refactor(chat-channel): separate ChatRepository and ChannelRepository, fix ViewModel architecture, enable private channel creation and mute system"

```
git add .  
git commit -m "feat(unread-server): move unreadCount increment to Cloud Function  
on message creation"
```

```
git add .  
git commit -m "feat(push-server): implement server-side push notifications respecting  
mutedUsers with Cloud Functions"
```

```
git commit -m "feat(security): strengthen Firestore rules for Spark plan with unread and  
members protection"
```

```
git add .  
git commit -m "refactor(architecture): enforce clean separation Activity -> ViewModel  
-> Repository for Spark version"
```

```
git commit -m "fix(firestore): restore channels and chat rules inside companies  
structure"
```

```
git add .  
git commit -m "refactor: migrate dashboard from apps collection to  
companies/businesses structure"
```

```
git add .  
git commit -m "fix: update Firestore rules to allow companies/businesses read by  
companyId"
```

```
git add .  
git commit -m "FASE 6 PRO - UnreadCount automático al enviar mensaje  
(client-side)"
```

```
git commit -m "fix: enable navigation to ChatActivity from AppAdapter"
```

```
git commit -m "fix: enable Chat, Updates and Private Channel buttons with proper  
navigation"
```

```
git commit -m "fix: enable Chat, Updates and add activity details"
```

```
git commit -m "feat: add AppDetailActivity and move Chat/Updates actions from  
dashboard to detail screen"
```

```
git commit -m "feat: add AppDetailActivity with ViewModel and business loading from  
Firestore"
```


git commit -m "feat: load business details and enable chat and updates from AppDetailActivity"

git commit -m "feat: add updates system per business with firestore integration"

fix(chat): correct Firestore message creation and repository logic

feat(chat): realtime chat messages using Firestore snapshot listener

feat(updates): admin panel can create project updates

feat(chat): add unread counter and mark chat as read

feat(chat): add panel Admin · create and publish posts · save update in Firebase · show result in UpdateActivity

git commit -m "refactor: migrate UpdatesAdapter to ListAdapter with DiffUtil"

git commit -m "feat: add unread message counter to chat channels · implent mark chat as read"

git commit -m "feat: add listen message and lastMessage · Save messages

git commit -m "Implement chat unread counters system"

git commit -m "feat: complete updates system CRUD"

git commit -m "feat: implement update System

git commit -m "feat: add pinned updates System"

git commit -m "feat: add unread updates counter"

git commit -m "feat: implement firebase push notifications"

Firebase

Usuarios

- cliente@test.com - TestClient
- admin@test.com - TestAdmin
- admin@ngd.com - NGDAdmin
- cliente@restaurante.com - RestClient

⚙️ FUNCIONAMIENTO FIREBASE - FIREBASE DATABASE

1 USERS → Quién entra en la app

Colección:

bash

 Copiar código

users

Aquí guardas **personas que inician sesión**.

Ejemplo documento (ID = UID de Authentication):

vbnet

 Copiar código

```
name: "Juan Pérez"
email: "admin@ngd.com"
role: "ADMIN"
companyId: "NGD_Studios"
businessId: "restaurante_madrid" (solo si es cliente)
```

👉 USERS = identidad + permisos

NO contiene datos del negocio.

2 COMPANIES → Empresas clientes

2 COMPANIES → Empresas clientes

Colección:

```
nginx
```

 Copiar código

```
companies
```

Documento ID:

```
nginx
```

 Copiar código

```
NGD_Studios
```

Campos:

```
vbnet
```

 Copiar código

```
name: "NGD Tech Solutions"
```

Subcolección:

```
nginx
```

 Copiar código

```
businesses
```

Aquí van los establecimientos de esa empresa.

Ejemplo:

```
markdown
```

 Copiar código

```
companies
```

```
  NGD_Studios
```

```
    businesses
```

```
      restaurante_madrid
```

```
      restaurante_valencia
```

Cada business tiene:

```
nginx
```

 Copiar código

```
name
```

```
status
```

```
progress
```

```
version
```

```
lastUpdate
```

👉 COMPANIES = estructura empresarial

👉 BUSINESSES = apps reales que estás desarrollando



3 UPDATES → Noticias o cambios

3 UPDATES → Noticias o cambios

Colección:

`nginx` Copiar código

`updates`

Sirve para:

- Novedades
- Avisos
- Cambios importantes
- Comunicados

Ejemplo documento:

`vbnet` Copiar código

```
title: "Nueva versión 1.2"
description: "Se mejora rendimiento"
companyId: "NGD_Studios"
date: "17/02/2026"
```

👉 UPDATES = noticias generales o por empresa

4 APPS → ¿Para qué sirve?

Aquí está la confusión.

En tu proyecto actual...

! `apps` sobra.

Porque la información real de la app ya está dentro de:

`nginx` Copiar código

`companies` → `businesses`

La colección `apps` solo tendría sentido si:

- Quisieras un catálogo global de apps
- O apps compartidas entre empresas
- O un sistema SaaS muy complejo

Pero para tu modelo actual de soporte:

👉 No la necesitas.

FUNCIONAMIENTO FIREBASE - AÑADIR USUARIO Y CLIENTES O EMPRESAS

PASO 1 — Crear empresa

En Firestore:

```
javascript
```

 Copiar código

```
companies → Add document
```

ID:

```
nginx
```

 Copiar código

```
empresa_pepe
```

Campo:

```
vbnet
```

 Copiar código

```
name: "Empresa Pepe SL"
```

PASO 2 — Crear negocios de esa empresa

Dentro de:

```
nginx
```

 Copiar código

```
companies → empresa_pepe → businesses
```

Añadir documento:

```
nginx
```

 Copiar código

```
restaurante_madrid
```

Campos:

```
nginx
```

 Copiar código

```
name
```

```
status
```

```
progress
```

```
version
```

```
lastUpdate
```

● PASO 3 — Crear usuario en Authentication

Authentication → Add user

Email + password

Copiar UID.

● PASO 4 — Crear documento en USERS

En:

```
bash
```

 Copiar código

```
users → Add document
```

ID = UID

Campos:

Para admin empresa:

```
pgsql
```

 Copiar código

```
name
email
role = ADMIN
companyId = empresa_pepe
```

Para cliente:

```
java
```

 Copiar código

```
name
email
role = CLIENT
companyId = empresa_pepe
businessId = restaurante_madrid
```





ORDEN IMPORTANTE

Primero:

- ✓ companies
- ✓ businesses

Después:

- ✓ Authentication
- ✓ users

Nunca al revés.



RESUMEN SIMPLE

Colección	Sirve para
users	Personas que entran
companies	Empresas clientes
businesses	Apps/proyectos de cada empresa
updates	Noticias
apps	(No necesaria ahora)

CUANDO CREAS UN NUEVO USUARIO (CLIENTE O ADMIN)

Siempre debes hacer 2 cosas en Firebase:

1. Crear el usuario en Authentication
2. Crear su documento en Firestore correctamente vinculado

PASO 1 — CREAR USUARIO EN AUTHENTICATION

Ir a Firebase Console

Build → Authentication → Users

Click en: Add user

Completar:

- Email
- Password (mínimo 6 caracteres)

Click **Add user**

IMPORTANTE

Después de crearlo:

👉 Copia el UID que aparece.

Ese UID es la clave que conecta todo.

PASO 2 — CREAR DOCUMENTO EN FIRESTORE

Build → Firestore Database

Colección: **users**

Add document

Document ID

👉 Pega el UID del usuario que acabas de crear.

● ESTRUCTURA ADMIN Y CLIENTE

◆ Campos que DEBE tener

Para ADMIN:

```
typescript
name → string → Juan Admin
email → string → admin@empresa.com
role → string → ADMIN
companyId → string → company_ngd
```

Copiar código

Para CLIENTE de un negocio:

```
typescript
name → string → Restaurante Madrid
email → string → cliente@restaurante.com
role → string → CLIENT
companyId → string → company_ngd
businessId → string → restaurante_madrid
```

Copiar código

Entonces:

- Si el usuario tiene companyId correcto → verá los negocios de esa empresa
- Si no tiene companyId → no verá nada

🧠 CASOS REALES

 CASO 1 — Nuevo ADMIN de la empresa

Solo necesita:

```
ini
role = ADMIN
companyId = company_ngd
```

Copiar código

Y automáticamente verá todos los negocios de esa empresa.

 CASO 2 — Nuevo CLIENTE para un negocio específico

Necesita:

```
ini
role = CLIENT
companyId = company_ngd
businessId = restaurante_valencia
```

Copiar código

Y verá solo ese negocio.



MEJORARLO PROFESIONALMENTE (RECOMENDADO)

Ahora mismo lo haces manualmente en Firebase.

Lo profesional sería:

👉 Que cuando el ADMIN cree un usuario desde la APP, automáticamente se cree en:

- Authentication
- Firestore
- Con role y companyId correctos

Eso lo haremos más adelante con:

- Cloud Functions
- O panel admin interno

Pero por ahora el manual está bien.